

Prompt Claude - Initialisation et construction parallèle de NotifyAfrica

NotifyAfrica

22 septembre 2026

PROMPT À FOURNIR À CLAUDE

Tu vas initialiser et développer l'écosystème **NotifyAfrica** à partir du Design System et des maquettes Claude Design que je vais te fournir.

Tu dois lire intégralement les documents de spécifications qui accompagnent ce prompt avant de créer le code :

1. 00_Contexte_Global_NotifyAfrica.md
2. 01_Specifications_Website_NotifyAfrica.md
3. 02_Specifications_Backoffice_NotifyAfrica.md
4. 03_Specifications_Console_NotifyAfrica.md

Ces documents sont autoritaires pour la logique fonctionnelle. Les maquettes et le Design System sont autoritaires pour l'apparence, les composants, la hiérarchie visuelle et les interactions déjà validées.

1. OBJECTIF

Construire en parallèle :

- NotifyAfrica Website ;
- NotifyAfrica Admin ;
- NotifyAfrica Console ;
- NotifyAfrica Core API nécessaire pour relier les trois applications.

Les trois interfaces doivent être fonctionnelles dans le même environnement de développement et déployables indépendamment sur des domaines différents.

2. STACK IMPOSÉE

Utilise :

- Node.js ;

- Next.js ;
- TypeScript ;
- backend également construit avec Next.js en runtime Node ;
- PostgreSQL recommandé ;
- Redis si nécessaire pour cache/queue ;
- Docker ;
- architecture compatible VPS.

Ne crée pas un système de microservices complexe au démarrage.

Utilise un **monorepo modulaire**.

Structure recommandée :

```

notifyafrica/
  apps/
    website/
    console/
    admin/
    core-api/
    worker/
  packages/
    design-system/
    ui/
    types/
    validation/
    api-client/
    config-client/
    config-seed/
    auth/
    observability/
  infra/
    proxy/
    docker/
    scripts/
  docs/

```

Tu peux adapter cette structure si une justification technique forte l'exige, mais conserve la séparation fonctionnelle et la capacité de déploiement indépendant.

3. DÉVELOPPEMENT EN PARALLÈLE

Ne termine pas entièrement une application avant de commencer les autres.

Travaille par tranches verticales partagées.

Ordre conseillé :

Phase A - Foundation

Construire simultanément :

- monorepo ;
- Design System ;
- types partagés ;
- Core API ;
- Auth ;
- Config Registry ;
- Catalog ;
- seed ;
- shells Website/Admin/Console.

Phase B - Configuration dynamique

Construire simultanément :

- Admin Catalog ;
- Admin Pricing ;
- Website Catalog/Pricing ;
- Console Catalog/Pricing ;
- invalidation/cache.

Phase C - Accounts

- inscription Console ;
- login ;
- organisation ;
- projet ;
- Admin Users/Organizations ;
- notification USER_REGISTERED.

Phase D - Wallet / SMS

- wallet ;
- pricing estimate ;
- SMS simple ;
- Admin transactions ;
- Website pricing.

Puis continuer lot par lot selon les spécifications.

4. DESIGN SYSTEM

Je vais te fournir un design déjà réalisé dans Claude Design.

Tu dois :

1. l'auditer ;
2. inventorier les composants ;
3. réutiliser les tokens ;
4. créer un package partagé ;
5. éviter de répliquer les composants entre applications.

Si une maquette contient une donnée métier d'exemple, ne transforme pas cette valeur en constante locale.

5. RÈGLE ABSOLUE : RIEN DE MÉTIER EN DUR

Aucun prix, pays, opérateur, remise, marge, produit, offre, limite, quota, devise, provider ou texte tarifaire ne doit être dupliqué dans les composants.

Les composants doivent consommer :

- Core API ;
- Config Client ;
- Catalog ;
- Pricing Engine.

6. FALLBACK / SEED

Point essentiel : au tout premier lancement, le Backoffice peut être vide.

Dans ce cas, le produit doit utiliser les valeurs fonctionnelles et contenus initiaux présents dans le design que je t'ai fourni.

Mais NE LES ÉCRIS PAS dans chaque composant.

Construis un package :

packages/config-seed

avec une configuration initiale structurée et versionnée.

Le mécanisme de résolution doit être :

```
Published Admin Config
  -> Scoped Override
  -> Design Seed
  -> Safe Empty Fallback
```

Le Core API doit retourner la valeur résolue.

Le Website et la Console ne doivent pas implémenter chacun leur propre fallback.

7. IMPORT DU SEED DANS ADMIN

Ajoute un mécanisme d'initialisation permettant au Backoffice d'importer la configuration seed.

L'import doit être :

- idempotent ;
- audité ;
- versionné ;
- non destructif.

8. DOMAINS

Prépare les applications pour être déployées séparément.

Exemples de cibles :

- Website : `notifyafrica.com`
- Console : `app.notifyafrica.com`
- Admin : `admin.notifyafrica.com`
- API : `api.notifyafrica.com`

Ces valeurs doivent venir des variables d'environnement et non être codées en dur.

9. WEBSITE -> CONSOLE

Le Website sert de porte d'entrée.

Tous les boutons **Créer un compte** doivent envoyer vers la Console.

Tous les boutons **Se connecter** doivent envoyer vers la Console.

La Console possède la logique d'authentification.

Conserve les paramètres utiles de campagne/source lors de la redirection.

10. ADMIN COMME SOURCE DE VÉRITÉ

Construis le Backoffice pour qu'il contrôle :

- produits ;
- offres ;
- pricing ;
- tranches ;
- discounts ;
- promotions ;
- pays ;

- devises ;
- opérateurs ;
- providers ;
- moyens de paiement ;
- content public ;
- feature flags ;
- devis ;
- routes ;
- proxy/gateway ;
- notifications.

11. PRICING INITIAL

Les données seed initiales incluent notamment :

SMS :

- 0-25 000 : 7 XOF/SMS ;
- 25 001-100 000 : 6,8 XOF/SMS ;
- volumes supérieurs : devis selon règles.

WhatsApp :

- tarifs provider officiels configurables ;
- marge initiale NotifyAfrica : +40 % ;
- markup configurable ;
- aucune constante métier dans le frontend.

Ces valeurs doivent devenir immédiatement modifiables depuis Admin.

12. SYNCHRONISATION

Lorsqu'un administrateur publie un changement :

- le Core API utilise la nouvelle version ;
- le Website se revalide ;
- la Console utilise la nouvelle règle ;
- les anciennes transactions conservent leur snapshot de prix historique.

13. NOTIFICATIONS INSCRIPTION

À chaque nouvelle inscription Console :

- créer `USER_REGISTERED` ;
- enregistrer l'événement ;
- déclencher les règles de notification ;
- afficher la notification Admin ;

- permettre email à Sales/Admin selon configuration.

Ne code pas les destinataires en dur.

14. PRICING ENGINE

Implémente un service central de pricing.

Entrée typique :

```
{
  "organizationId": "...",
  "projectId": "...",
  "product": "SMS",
  "country": "TG",
  "operator": null,
  "category": null,
  "quantity": 80000,
  "currency": "XOF"
}
```

Sortie typique :

```
{
  "unitPrice": 6.8,
  "subtotal": 544000,
  "discounts": [],
  "taxes": [],
  "total": 544000,
  "currency": "XOF",
  "ruleVersion": "...",
  "quoteRequired": false
}
```

Les chiffres ci-dessus illustrent un résultat possible du seed et ne doivent pas être codés dans la logique.

15. WALLET

Construis un ledger robuste.

Opérations :

- CREDIT ;
- DEBIT ;
- HOLD ;
- CAPTURE ;
- RELEASE ;

- REFUND ;
- ADJUSTMENT.

Pour les opérations payantes : estimation -> contrôle solde -> hold si nécessaire
-> exécution -> capture/release.

16. FUTUR AGRÉGATEUR

Même si les APIs paiement ne sont pas toutes développées immédiatement, prépare les modèles et l'Admin.

Construis des concepts génériques :

- Product ;
- Provider ;
- Operator ;
- ProviderEndpoint ;
- Route ;
- RequestMapping ;
- ResponseMapping ;
- ErrorMapping ;
- Credential ;
- ApiProduct ;
- Webhook ;
- Callback.

17. PROXY / GATEWAY

Le Backoffice doit permettre de configurer le futur proxy.

Ne crée pas un proxy arbitraire non sécurisé permettant d'appeler n'importe quelle URL.

Les destinations doivent être des providers approuvés et configurés.

Prévoir :

- environment ;
- base URL ;
- path ;
- auth ;
- timeout ;
- retry ;
- request mapping ;
- response mapping ;
- error mapping ;
- callback mapping ;
- route priority.

18. SECURITY

- secrets côté serveur ;
- encryption at rest ;
- API keys hashées ou sécurisées ;
- RBAC ;
- CSRF selon mécanisme auth ;
- validation des entrées ;
- rate limiting ;
- audit ;
- idempotency sur flux sensibles ;
- jamais de credentials providers dans Website/Console.

19. DATABASE

Utilise une base relationnelle pour :

- users ;
- organizations ;
- projects ;
- catalog ;
- pricing ;
- discounts ;
- wallets ;
- ledger ;
- transactions ;
- quotes ;
- providers ;
- routes ;
- notifications ;
- audit.

Chaque transaction financière doit conserver le pricing snapshot utilisé.

20. API CONTRACTS

Définis les contrats avec validation partagée.

Ne duplique pas les types entre apps.

Utilise un package partagé `types` / `validation`.

21. WORKERS

Pour les campagnes et callbacks, utilise un worker séparé.

Le frontend ne doit pas attendre la fin d'une campagne longue dans une requête HTTP.

22. VPS

Prépare Docker Compose pour le développement et le premier VPS.

Composants initiaux possibles :

- reverse proxy ;
- website ;
- console ;
- admin ;
- core-api ;
- worker ;
- postgres ;
- redis.

Ne lie pas le produit fonctionnellement à un unique VPS : chaque composant doit pouvoir être migré.

23. COMMANDES DÉVELOPPEMENT

Prévois une commande pour lancer tous les projets en parallèle.

Exemple attendu conceptuellement :

```
pnpm dev
```

qui lance Website, Console, Admin et Core API.

Prévois aussi :

```
pnpm dev:website  
pnpm dev:console  
pnpm dev:admin  
pnpm dev:api
```

24. QUALITÉ

Pour chaque tranche livrée :

- lint ;
- typecheck ;
- tests unitaires du Core ;
- tests d'intégration des règles critiques ;
- test de résolution config ;
- test pricing ;

- test wallet ;
- test RBAC ;
- test parcours signup.

25. ORDRE DE PREMIÈRE IMPLÉMENTATION

Commence par :

1. analyser le Design System et les maquettes ;
2. créer le monorepo ;
3. créer les shells des 3 applications ;
4. créer Core API ;
5. créer Config Registry + seed ;
6. créer Auth ;
7. créer Catalog ;
8. créer Pricing Engine minimal ;
9. brancher Website sur Catalog/Pricing ;
10. brancher Admin sur Catalog/Pricing ;
11. brancher Console sur Auth/Catalog/Pricing ;
12. implémenter `USER_REGISTERED` et notifications ;
13. seulement ensuite poursuivre Wallet/SMS/etc.

26. MÉTHODE DE TRAVAIL

Ne génère pas des dizaines de pages statiques avant que la couche de configuration existe.

À chaque étape :

1. implémente la source de données ;
2. crée la configuration Admin ;
3. expose le contrat Core API ;
4. consomme-le dans Website/Console ;
5. ajoute seed/fallback ;
6. teste ;
7. documente.

27. CRITÈRES DE FIN DE LA PREMIÈRE PHASE

La phase Foundation est terminée lorsque :

- `pnpm dev` lance les trois interfaces et l'API ;
- le design fourni est correctement intégré ;

- Website affiche un catalogue provenant de l'API ;
- Admin peut modifier une configuration puis la publier ;
- Website reflète le changement ;
- Console permet register/login ;
- un register crée une notification Admin ;
- seed fonctionne si la base de configuration est vide ;
- les domaines/URLs sont uniquement en configuration d'environnement ;
- aucun prix métier n'est codé dans un composant.

28. IMPORTANT

Si une ambiguïté mineure existe, prends une décision cohérente avec les spécifications et documente-la au lieu de bloquer l'implémentation.

Si le design et les spécifications semblent diverger :

- apparence et composants -> design fourni ;
- règles métier et données -> spécifications ;
- valeurs initiales absentes du Backoffice -> seed issu du design ;
- sécurité -> choisir l'option la plus sûre.

Commence maintenant par l'audit du design et l'initialisation du monorepo, puis construis les trois applications en parallèle conformément à ce plan.